

Skew, Shuffle und Spill erkennen und beheben

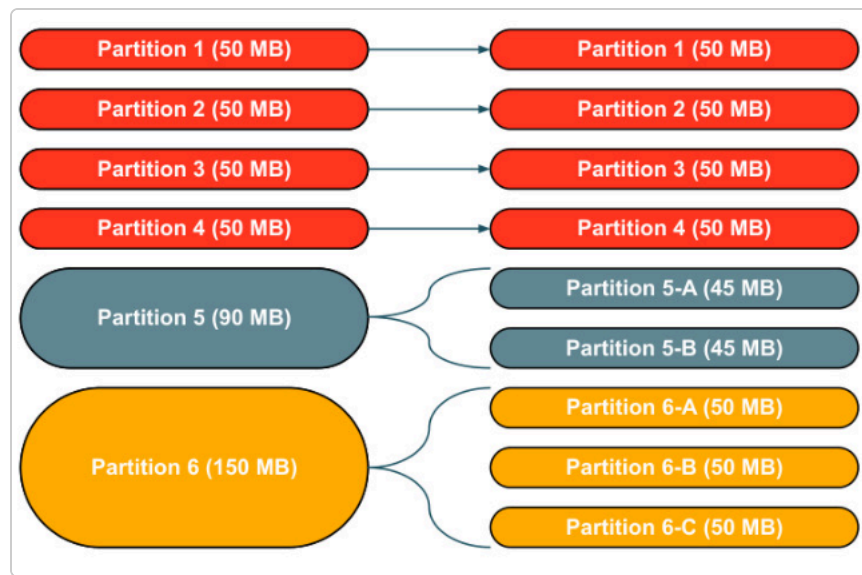
Section 6 · Troubleshooting, Monitoring and Optimization · Quelle: Kurs 7, Kapitel 3.1–3.3

Drei der häufigsten Ursachen für langsame oder sogar abstürzende Spark-Jobs sind **Data Skew** (ungleich verteilte Daten), **Shuffle** (Datenumverteilung über das Netzwerk) und **Spill** (Auslagern von Arbeitsspeicher auf die Festplatte). Alle drei Phänomene hängen eng zusammen: Skew verstärkt Shuffle-Probleme, und beide können zu Spill führen. Dieses Kapitel erklärt, wie die drei Effekte entstehen, wie man sie im Spark UI erkennt und mit welchen Maßnahmen man ihnen begegnet.

1. Data Skew: Wenn eine Partition alle anderen überragt

Beim initialen Einlesen werden Daten typischerweise in 128 MB große Partitionen aufgeteilt und gleichmäßig verteilt. Sobald jedoch transformiert wird – etwa durch eine Aggregation nach einer Spaltenüberschrift – kann eine Partition erheblich mehr Datensätze enthalten als eine andere. Ein einfaches Beispiel: Werden Transaktionsdaten nach Stadt aggregiert und eine Stadt hat doppelt so viele Einwohner wie die übrigen, landet in ihrer Partition auch doppelt so viel Datenvolumen. Ein geringer Grad an Skew ist meist unproblematisch, starker Skew führt jedoch zu Spill oder sogar zu schwer diagnostizierbaren **Out-of-Memory-Fehlern**.

Die Konsequenz von Skew: Da eine Stage erst abgeschlossen ist, wenn die *langsamste* Task fertig ist, bestimmt die am stärksten überladene Partition die Gesamtlaufzeit der gesamten Stage – und unter Umständen reicht der verfügbare Arbeitsspeicher für diese eine Partition nicht mehr aus.



Adaptive Query Execution erkennt überproportional große Partitionen zur Laufzeit und teilt sie automatisch in kleinere, gleichmäßiger verteilte Stücke auf.

Skew beheben: vier gestufte Strategien

1. **Skew-Werte herausfiltern** – Ist der Skew auf einen bestimmten (häufig: `NULL` -)Wert zurückzuführen, etwa bei einem Join über eine Spalte mit vielen Nullwerten, löst ein einfacher Filter das Problem oft vollständig.
2. **Skew-Hints** – Sind Tabelle, Spalte und idealerweise auch die betroffenen Werte bekannt, kann Spark über einen expliziten Hint mitgeteilt werden, dass hier Skew vorliegt, damit Spark das Problem gezielt behandeln kann.
3. **AQE-Skew-Optimierung** – Seit Spark 3.0 löst die Adaptive Query Execution Skew standardmäßig automatisch: Eine Partition gilt als „skewed“, wenn sie mindestens 256 MB groß und mindestens fünfmal größer als die durchschnittliche Partitionsgröße ist. Diese Schwellenwerte lassen sich konfigurieren. Bei mehr als 2.000 Shuffle-Partitionen kann AQE Skew allerdings nicht mehr zuverlässig erkennen, da Spark dann nur noch Durchschnittswerte statt exakter Shuffle-Block-Größen verfolgt – hier hilft entweder eine Reduzierung der Shuffle-Partitionen unter 2.000 oder eine Anpassung der entsprechenden Spark-Konfiguration.
4. **Salting** – Führt keine der vorherigen Maßnahmen zum Erfolg, bleibt als letztes Mittel das Salting: Den Werten der Skew-Spalte werden zufällige Suffixe angehängt, um eine große, ungleich verteilte Partition künstlich in mehrere kleinere, gleichmäßig verteilte Partitionen aufzuteilen.

2. Shuffle: Datenumverteilung über das Netzwerk

Ein **Shuffle** tritt immer dann auf, wenn Daten am Ende einer Stage neu über das Netzwerk verteilt werden müssen, damit die nächste Stage darauf aufbauen kann – klassischerweise bei Operationen wie `join()`, `distinct()`, `groupBy()`, `orderBy()` oder bestimmten Aktionen wie `count()`. Solche Operationen werden als **Wide Transformations** bezeichnet, weil sie zwei Stages benötigen: In der ersten Stage (Map) wird vorbereitet, welche Daten wohin verschoben werden müssen; anschließend erfolgt der eigentliche Shuffle über das Netzwerk; in der zweiten Stage (Reduce) werden die neu angeordneten Daten weiterverarbeitet. Operationen ohne Shuffle heißen entsprechend **Narrow Transformations** und benötigen nur eine einzige Stage.

Shuffle-Strategien bei Joins

Welche konkrete Join-Strategie Spark wählt, hat großen Einfluss auf die Menge an Shuffle-Daten:

- **Broadcast Hash Join** – Die kleinere Tabelle wird vollständig an alle Executor verteilt, wodurch ein teurer Shuffle der großen Tabelle entfällt. Ideal, wenn eine der beiden Tabellen klein genug in den Speicher passt.
- **Shuffle Hash Join** – Standard-Join-Strategie bei Databricks Photon.
- **Sort-Merge Join** – Standard-Join-Strategie im Open-Source-Spark ohne Photon.

Shuffle reduzieren: Empfehlungen

- Wenige, dafür größere virtuelle Maschinen (mehr Cores) verwenden – die Kosten für Festplatten-I/O bleiben zwar bestehen, das Netzwerk-I/O sinkt jedoch.
- Auf AQE und Dynamic Partition Pruning (DPP) verlassen, statt Datensätze manuell zu denormalisieren – beide Techniken machen viele frühere Denormalisierungs-Workarounds überflüssig.
- Die zu shuffelnde Datenmenge reduzieren: unnötige Spalten frühzeitig entfernen, nicht benötigte Datensätze vorab herausfiltern.
- **Bucketing** nur mit Bedacht einsetzen – es eliminiert zwar den Sortierschritt im Sort-Merge-Join durch vorsortierte Partitionen, ist aber aufwendig korrekt umzusetzen und lohnt sich in der Regel erst ab Datenmengen von mehreren Terabyte.

3. Spill: Wenn der Arbeitsspeicher nicht ausreicht

Spill beschreibt, was passiert, wenn eine Datenmenge nicht mehr vollständig in den verfügbaren Arbeitsspeicher (RAM) passt: Ein Teil der Daten wird auf die Festplatte ausgelagert und bei Bedarf später wieder zurückgelesen. Da Festplattenzugriffe deutlich langsamer sind als Arbeitsspeicherzugriffe, verlangsamt Spill die Verarbeitung erheblich – findet dieser

Auslagerungsprozess gar nicht erst statt und reicht der Speicher trotzdem nicht aus, kommt es zu einem Out-of-Memory-Fehler, der den gesamten Job zum Absturz bringt.

Typische Ursachen für Spill

- `spark.sql.files.maxPartitionBytes` zu hoch eingestellt (Standardwert: 128 MB) – führt zu überdimensionierten Partitionen.
- `explode()` selbst kleiner Arrays, wodurch deutlich mehr Zeilen entstehen als vorher vorhanden waren.
- `join()` oder `crossJoin()` zweier Tabellen, die eine große Zahl neuer Zeilen erzeugen – insbesondere bei Cross-Joins oder Joins auf Skew-Schlüsseln.
- `groupBy()` auf Spalten mit niedriger Kardinalität, `countDistinct()` oder `collect_set()`.
- `spark.sql.shuffle.partitions` zu niedrig gesetzt oder falsche Verwendung von `repartition()`.

Spill im Spark UI erkennen

Spill wird ausschließlich auf der Detailseite einer einzelnen Stage angezeigt (in den zusammengefassten Metriken, den nach Executor aggregierten Metriken oder in der Tasks-Tabelle) bzw. in den entsprechenden Query-Details. Das macht Spill leicht zu übersehen – die Spalten für „Spill (Memory)“ und „Spill (Disk)“ erscheinen im UI überhaupt nur, wenn tatsächlich Spill vorliegt. Sind diese Spalten sichtbar, ist das bereits ein eindeutiges Warnsignal. Der Wert „Spill (Disk)“ ist dabei stets kleiner als „Spill (Memory)“, da die Daten beim Schreiben auf die Festplatte automatisch komprimiert werden.

Spill vermeiden: Gegenmaßnahmen

- Cluster mit mehr RAM pro Core bereitstellen.
- Data Skew beheben (siehe oben) – oft ist Skew die eigentliche Ursache eines Spill-Problems.
- Größe der Spark-Partitionen gezielt steuern.
- Teure Operationen wie `explode()` möglichst vermeiden.
- Datenmenge frühzeitig reduzieren, wo immer möglich.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks empfiehlt, Skew-Probleme grundsätzlich vor Spill-Problemen zu beheben, da Skew häufig die eigentliche Ursache eines Spills ist. Als schnelle Diagnose gilt: Ist die maximale Task-Dauer einer Stage mehr als 50 % höher als die Dauer des 75. Perzentils, liegt vermutlich Skew vor. Die „Skew and Spill“-Seite des Spark-UI-Guides

beschreibt detailliert, welche Metriken auf der Stage-Detailseite zu prüfen sind.

Quelle: [Skew and spill – Databricks-Dokumentation](#)